

Claude for BlackBerry OS 6 — Client & From-Scratch TLS 1.2 Stack

As of 2026-10-06 · Bilal Ahmed · Draft

Overview

Claude for BlackBerry is a native app that runs a full Claude API client on a BlackBerry Torch 9800 under BlackBerry OS 6 (and runs on OS 5 and 7 as well). It is a real RIM `UiApplication` with custom-painted screens, not a J2ME MIDlet.

The target hardware is a 2010 smartphone: a 624 MHz processor, a 360x480 touchscreen, a physical keyboard and trackpad, and a Java runtime based on CLDC 1.1 and the Java 1.3 language. The core problem is that this device cannot reach `api.anthropic.com` by any normal route. Its built-in networking and TLS are far too old for a modern HTTPS server, and the usual fix — calling a newer crypto library — is closed off on BlackBerry because the APIs that would do it require a code-signing key from RIM, and RIM stopped issuing those keys and shut down the signing servers.

The app solves this by carrying everything it needs itself: its own HTTP client, its own TLS 1.2 implementation written from scratch in Java, and its own certificate trust store. It uses no signed RIM APIs at all, which is what lets it install and run unsigned on a real device.

This document describes how the app is built, with the from-scratch TLS stack as the centrepiece, and records the constraints that shaped it and the decisions made along the way.

Why it had to be built this way

Three facts about the platform forced the design, and they stack on top of each other.

The device's own TLS is too old. BlackBerry OS 6 shipped before modern TLS. Its built-in HTTPS support tops out at protocol versions and cipher suites that `api.anthropic.com` no longer accepts, so a normal secure connection from the device is refused during the handshake. There is no setting that fixes this; the code to speak modern TLS simply isn't there.

RIM's crypto APIs are locked behind signing. BlackBerry does ship cryptographic classes that could, in principle, do modern TLS. But they are "controlled" APIs: any app that calls them must be signed with a key issued by RIM's signing server. RIM wound that service down, so new signatures can't be obtained. An app built on those APIs would therefore refuse to load on a real device.

So the app uses no signed APIs at all. Rather than depend on anything RIM controls, the app carries its own implementation of every piece it needs: the TLS handshake, the ciphers, the certificate checks and the trust store. Everything is built on the plain, unsigned subset of the platform — sockets, drawing, input and file storage. Because nothing it calls is controlled, the app installs and runs unsigned, with no RIM key involved.

The cost of this independence is that the hardest part of the project — a complete TLS 1.2 client — had to be written by hand in an old dialect of Java. That is the subject of the next two sections.

Architecture

The app mirrors the structure of the other Claude Classic clients. Those are written in C around a shared core; BlackBerry can't run C, so the core was re-implemented in Java 1.3, keeping the same shape and the same on-disk formats so a chat saved on one client reads on another.

The code is organised into three layers, all under `com.claudeclassic.bb`:

bullet `tls/` — **the transport**. A from-scratch TLS 1.2 client: the handshake state machine, elliptic-curve key exchange (`Ec`), the record layer and AES-128-GCM, and certificate verification. It sits directly on a plain socket and presents an ordinary encrypted stream to the layer above.

bullet `core/` — **the portable logic**. The Java port of the shared core: `Http` (HTTP/1.1 over the TLS stream), `Stream` (the server-sent-events parser and the Messages API stream accumulator), `Api` (building requests and parsing the model list), `Json`, `Utf8`, `Base64`, `Md` (Markdown to a display model) and `Store` (chat and settings storage).

bullet **The UI classes — the RIM front end**. A `UiApplication` with custom-painted screens: `ChatScreen`, `ChatListScreen`, `Composer`, `SettingsScreen`, `TitleBar`, `PickerScreen`, `PromptScreen`, `TextScreen`, `FilePicker`, and support classes (`Theme`, `Layout`, `Images`, `MessageField`, and the greeting and error fields).

The flow is straightforward: a UI screen asks `core/Api` to build a request; `core/Http` sends it over a connection from `tls/`; the server's streamed reply comes back up through `Stream`, is turned into display blocks by `Md` and `cc_render`'s Java equivalent, and is painted by the chat screen as it arrives. Only the UI layer is BlackBerry-specific; everything below it is portable and shared in spirit with the Mac, Windows and iPhone builds.

The TLS 1.2 stack, from scratch

This is the heart of the project: a working TLS 1.2 client written by hand in Java 1.3, with no crypto library underneath it. It does enough of the protocol to talk to `api.anthropic.com` securely, and nothing more.

The handshake. The client runs a full TLS 1.2 handshake over a plain TCP socket: it sends a ClientHello offering the protocol version, cipher suite and supported curves; receives the server's ServerHello, certificate chain and key-exchange parameters; verifies the chain; completes the key exchange; and switches both directions to encrypted records. From there the socket carries an ordinary byte stream that the HTTP layer uses without knowing TLS is beneath it.

Key exchange (`Ec`). The stack implements Elliptic Curve Diffie-Hellman (ECDHE) over three curves — X25519, NIST P-256 and P-384 — so it can agree a shared secret with whichever curve the server picks. The elliptic-curve maths is implemented from the ground up, including the big-integer and field arithmetic it rests on, because none of it can come from a signed API.

Bulk encryption. Once keys are agreed, records are protected with AES-128-GCM, which provides both confidentiality and integrity. The AES block cipher and the GCM mode are both implemented in the stack.

Certificate verification. The server's certificate chain is checked rather than trusted blindly: signatures are verified with RSA and ECDSA, and the chain is validated up to a trusted root. This is what stops a man-in-the-middle from impersonating the API, and it is the part that makes the connection genuinely secure rather than merely encrypted.

The trust store (`res/roots.bin`). Instead of relying on the device's outdated certificate store, the app ships its own. `roots.bin` is a compact binary bundle of the Mozilla root certificates, built on a computer by `tools/make_bb_roots.py` and loaded at startup. Shipping the roots also means the trust anchors can be refreshed by rebuilding that file, independent of the device's age.

The practical cost is time: on a 624 MHz Torch the handshake takes a few seconds, since the public-key maths is pure interpreted Java. That is a one-time cost per connection, and an optional desktop relay (`tools/bb_relay.py`) exists for anyone who finds it slow on home Wi-Fi, though it isn't needed for the app to work.

The application layer

With a secure stream available, the rest is a complete chat client, ported from the shared core.

- HTTP and streaming.** `Http` speaks HTTP/1.1 over the TLS stream, including proxy CONNECT. `Stream` parses the server-sent-events response and accumulates the Messages API stream, so replies render token by token as they arrive rather than all at once.

- The Messages API.** `Api` builds requests with the full set of options — model selection, thinking, effort, web search and prompt caching — and parses the model list returned by the API.

- Markdown rendering.** Replies are rendered with headings, lists, task lists, quotes and code blocks. Tables are drawn as cards rather than true grids, which also sidestepped an early crash. Thinking and web-search steps show as their own rows, with citations and sources.

- Composing and attachments.** The composer has model, thinking and search chips, and can attach photos, PDFs and text files picked from the media card.

- Conversation management.** Retry, edit-last-message, rename, delete and automatic titles all work.

- Settings.** Style, custom preferences, effort, maximum length, text size, connection type and APN are all adjustable.

- Storage.** Chats are saved to `/store/home/user/ClaudeClassic/` in the same shared format the other clients use.

Input is handled for all three methods the Torch offers: the trackpad, the physical keyboard and the touchscreen. Enter sends and Shift+Enter inserts a newline, Escape stops a running reply, and every action is also reachable from the BlackBerry menu. Copying text uses a dedicated read-only text screen, because the clipboard API is one of the controlled, signing-required APIs the app avoids.

Constraints and workarounds

The environment imposed a set of limits, each with a specific fix.

- Java 1.3 only.** No generics, no enhanced-for, no autoboxing, none of the conveniences added after 1.3. The code is written to that language level throughout, and a custom checker enforces it so a modern syntax slip can't slip through.

- No signed APIs.** Everything that would normally be a one-line library call — crypto, clipboard — had to be done another way: the TLS stack for crypto, a read-only text screen for copying.

- Missing font glyphs.** The BlackBerry fonts don't include the checkbox glyphs the UI needed, so the checkboxes are drawn by hand rather than printed as characters.

- bullet **Custom-painted UI with protected calls.** Some of RIM's screen-refresh methods (`invalidate()`) are protected; the app wraps them in its own `refresh()` helpers to drive repaints.
- bullet **The one-row-table crash.** An early version rendered a streaming table as a one-row grid, which threw a `ClassCastException`. The fix was to flatten the cells, which is also why tables now render as cards.
- bullet **Non-ASCII in source.** Non-ASCII characters written directly into the source files caused trouble, so every one is written as a `\u` escape instead.

None of these were show-stoppers on their own, but together they're the reason the app is as much a platform-archaeology exercise as a coding one: much of the work was finding what the unsigned subset of an old, locked-down platform actually allows, and staying inside it.

Build and tooling

Building the app. On Windows with the BlackBerry JDE 6.0 and a 32-bit JDK 6, `blackberry\build.bat` compiles the sources at Java 1.3 source level against `net_rim_api.jar`, packages the classes with the PNG resources and `roots.bin` into a jar, and runs RIM's `rapc` compiler to produce `build\Claude.cod` and a `Claude.alx` for Desktop Manager. On a Mac or Linux machine that has `rapc.jar`, `blackberry/build.sh <jde folder>` does the same. No signing step is needed, because the app uses no controlled APIs.

Checking without RIM tools. Because the real toolchain only runs on specific setups, there are two checks that run anywhere:

- bullet `blackberry/tools/check_bb.sh` compiles everything against generated signature stubs for CLDC 1.1 and the BlackBerry API, so the code is verified to call only APIs that exist. The stubs are generated from spec files (`cldc_spec.txt`, `rim_spec.txt`) by `gen_stubs.py`.
- bullet `Java13Check.java` is a tree checker that walks the source and rejects any syntax newer than Java 1.3, catching modern constructs the stub compile might otherwise allow.

The desktop simulator. `blackberry/sim/` is a desktop stand-in for the RIM UI classes, and `blackberry/test/run_sim.sh` runs the actual app on it, rendering Torch-sized screens. This is how the screenshots in `blackberry/screenshots/` were produced, and how the UI was iterated without flashing a device each time. It also surfaced several bugs of its own during development — event-loop and escape-key handling, and nested UI calls — which were fixed there before they could reach hardware.

There is also a small test suite (`CoreTest`, `TlsTest`) covering the core and the TLS stack, with fixtures generated by `make_fixture.py`.

Install and first run

Installing. With the Torch connected over USB, the app loads with `JavaLoader -usb load Claude.cod`, through Desktop Manager using `Claude.alx`, or via the BlackBerry Java Plug-in for Eclipse by importing `src/` and `res/` as a BlackBerry Application. No signing is required.

First run. The first time it connects and saves, the app asks for Connections and Files permission; both must be allowed (or set afterward in Options > Applications > Claude > Edit Permissions). The API key

is then pasted into Settings — pasting rather than typing, since the on-screen flow is faster than the keyboard — followed by *Test Key & Load Models*. *Connection Automatic* tries Wi-Fi first, then the mobile network as a direct TCP connection, with an APN setting for carriers that need one.

Testing status. The app has been exercised thoroughly on the desktop simulator, which is where the screenshots and most bug-fixing came from. As of this draft it has **not yet been confirmed on a physical Torch 9800** — that is the next step. The most important thing to watch on real hardware is the TLS handshake: whether it completes against api.anthropic.com, and how long it takes on the 624 MHz device. The simulator runs the same TLS code, so a successful simulator handshake is a good sign, but only the real radio and processor will confirm it end to end.

This section should be updated with what actually happens on the device: handshake success and timing, whether permissions behave as expected, and anything that renders or performs differently from the simulator.

Open questions and running log

Open questions

- Does the from-scratch handshake complete against api.anthropic.com on a real Torch, and how many seconds does it take there?
- Is the few-second handshake acceptable in practice, or is the optional [bb_relay.py](#) worth recommending for regular use?
- Do streaming replies keep up on the 624 MHz device, or does rendering lag behind the network?
- Any rendering differences between the simulator and real hardware, especially the hand-drawn elements and the table cards?

Running log

Keep a short dated line here each session — what was tried, what happened, and dead ends especially. This is the raw material for the eventual write-up, and the details are easy to forget later.

- 2026-10-06: Draft documentation started. App complete and working on the desktop simulator; not yet run on a physical Torch. Fixes to date: one-row-table `ClassCastException` (flattened cells, tables now render as cards), non-ASCII escaped to `\u` throughout, checkboxes hand-drawn for missing glyphs, `invalidate()` wrapped in `refresh()` helpers, and several simulator-only bugs (escape handling, event-loop crash, nested UI calls).